

Top 10 Scenario-Based PySpark Interview Questions

1. Remove Duplicate Records and Keep the Latest Record

- **Question:** You have multiple records for the same customer. Keep only the **latest record based on updated_at**.
- **Approach:** Use `ROW_NUMBER()` with a Window partitioned by the business key and ordered by `updated_at DESC`.
- **Theory:** `row_number()` assigns a sequential number to each row within a window partition. The latest record gets `rn = 1`.
- **Mistakes:** Using `distinct()`; using `dropDuplicates()` without controlling which record is retained.
- **Final Query:**

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

window_spec = Window.partitionBy("customer_id") \
    .orderBy(col("updated_at").desc())

result = (
    df.withColumn("rn", row_number().over(window_spec))
      .filter(col("rn") == 1)
      .drop("rn")
)
```

LinkedIn@AshishZope

2. Find the Second-Highest Salary in Each Department

- **Question:** Find the employee having the **second-highest salary in each department**.

- **Approach:** Use `DENSE_RANK()` partitioned by department and ordered by salary descending.
- **Theory:** `dense_rank()` gives the same rank to employees having the same salary without leaving gaps in ranking.
- **Mistakes:** Using `row_number()` when salary ties need to be considered.
- **Final Query:**

LinkedIn@AshishZope

```
from pyspark.sql.window import Window
from pyspark.sql.functions import dense_rank, col

window_spec = Window.partitionBy("department") \
                    .orderBy(col("salary").desc())

result = (
    df.withColumn("rank", dense_rank().over(window_spec))
      .filter(col("rank") == 2)
      .drop("rank")
)
```

3. Find Customers Who Have Not Placed Any Orders

- **Question:** You have `customers` and `orders` DataFrames. Find customers who **have never placed an order**.
- **Approach:** Use a **Left Anti Join**.
- **Theory:** `left_anti` returns records from the left DataFrame for which **no matching record exists** in the right DataFrame.
- **Mistakes:** Using a left join and forgetting to filter NULL values; using `inner` join.
- **Final Query:**

```
result = customers.join(
    orders,
    customers.customer_id == orders.customer_id,
    "left_anti"
)
```

4. Handle a Data Skew Problem During a Join

- **Question:** A PySpark job is taking too long because one `customer_id` contains millions of records and the join is suffering from **data skew**. How would you optimize it?
- **Approach:** Identify the skewed key and use **Salting** or a **Broadcast Join** when the other DataFrame is small.
- **Theory:** Data skew occurs when data is distributed unevenly across partitions. One executor receives significantly more data than others, causing a **straggler task**.
- **Mistakes:** Increasing executor memory blindly; simply increasing the number of partitions.
- **Final Query:**

```
from pyspark.sql.functions import broadcast

result = large_df.join(
    broadcast(small_df),
    "customer_id",
    "inner"
)
```

Interview Point: Broadcast join is useful when one side of the join is sufficiently small to fit in executor memory.

5. Process a 500 GB File Without Running Out of Memory

- **Question:** You receive a **500 GB CSV file** every day. How would you process it efficiently using PySpark?
- **Approach:** Read the data using Spark's distributed processing instead of loading it into Python memory. Select only required columns, filter early, use an efficient format such as Parquet/Delta, and control partitioning.
- **Theory:** Spark processes large datasets across multiple executors. It does **not require the complete dataset to fit into the driver's memory**.
- **Mistakes:** Using `collect()`; converting the entire DataFrame to Pandas using `toPandas()`.
- **Final Query:**

```
df = (
    spark.read
        .option("header", True)
        .csv("/input/customer_data/")
)

result = (
    df.select(
        "customer_id",
        "name",
        "salary"
    )
    .filter(col("salary") > 50000)
)

result.write \
    .mode("overwrite") \
    .parquet("/output/customer_data/")
```

LinkedIn@AshishZope

Interview Point: Avoid unnecessary data movement to the Driver.

6. Incremental Load Using Last Modified Date

- **Question:** You have a source table containing `last_modified_date`. Every day you need to process **only newly inserted or modified records** instead of the entire dataset. How would you implement this?
- **Approach:** Maintain a **watermark** containing the last successfully processed timestamp.
- **Theory:** Incremental processing reduces the amount of data read and processed compared with a full load.
- **Mistakes:** Hardcoding the date; updating the watermark before the job successfully completes.
- **Final Query:**

```
last_watermark = "2026-09-05 23:59:59"

incremental_df = source_df.filter(
    col("last_modified_date") > last_watermark
)
```

LinkedIn@AshishZope

After successful processing:

```
new_watermark = (
    incremental_df
    .agg({"last_modified_date": "max"})
    .collect()[0][0]
)
```

LinkedIn@AshishZope

Interview Point: In production, store the watermark in a control/metadata table rather than hardcoding it.

7. Calculate Running Total for Each Customer

- **Question:** Calculate the **running transaction amount** for each customer ordered by transaction date.

- **Approach:** Use a Window function with `partitionBy(customer_id)` and an ordered window.
- **Theory:** A running total calculates the cumulative value of a measure as rows progress according to a defined ordering.
- **Mistakes:** Using `groupBy()` because it produces only one aggregated row per customer.
- **Final Query:**

LinkedIn@AshishZope

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum, col

window_spec = (
    Window
    .partitionBy("customer_id")
    .orderBy("transaction_date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
)

result = df.withColumn(
    "running_total",
    sum(col("amount")).over(window_spec)
)
```

8. Find the Top 3 Products by Sales in Each Category

- **Question:** Find the **top 3 products based on total sales within every category**.
- **Approach:** First aggregate sales by category and product, then apply `ROW_NUMBER()` or `DENSE_RANK()`.
- **Theory:** Window functions allow ranking **within each group without collapsing the rows like `groupBy()` does**.
- **Mistakes:** Applying `limit(3)` because that returns only three records globally.
- **Final Query:**

```
from pyspark.sql.functions import sum, row_number
from pyspark.sql.window import Window

sales = (
    df.groupBy("category", "product")
        .agg(sum("sales").alias("total_sales"))
)

window_spec = (
    Window
        .partitionBy("category")
        .orderBy(col("total_sales").desc())
)

result = (
    sales.withColumn(
        "rn",
        row_number().over(window_spec)
    )
    .filter(col("rn") <= 3)
    .drop("rn")
)
```

9. Explode Nested JSON Data

- **Question:** Your DataFrame contains an array of orders for every customer. Convert each order into a **separate row**.
- **Approach:** Use the `explode()` function.
- **Theory:** `explode()` converts each element of an array/map into a separate row.
- **Mistakes:** Treating an array column like a normal scalar column; using Python loops.
- **Final Query:**

```
from pyspark.sql.functions import explode, col

result = df.withColumn(
    "order",
    explode(col("orders"))
)
```

If `orders` contains structs:

```
result = (
    df.withColumn("order", explode("orders"))
    .select(
        "customer_id",
        col("order.order_id").alias("order_id"),
        col("order.amount").alias("amount")
    )
)
```

10. Optimize a Slow PySpark Job

- **Question:** A PySpark job that normally takes **20 minutes suddenly takes 2 hours**. How would you troubleshoot and optimize it?
- **Approach:** Check the Spark UI and identify:
 - Large shuffles
 - Data skew
 - Excessive partitions
 - Too few partitions
 - Expensive joins
 - Unnecessary transformations
 - Repeated DataFrame calculations
 - Poor file formats
 - Small-file problem

- **Theory:** PySpark performance depends heavily on **partitioning, shuffles, joins, serialization, memory usage, and data distribution**.
- **Mistakes:** Immediately increasing executor memory or cores without identifying the bottleneck.
- **Final Query:**

LinkedIn@AshishZope

```
# Select only required columns
df1 = df1.select(
    "customer_id",
    "amount"
)

# Filter early
df1 = df1.filter(
    col("amount") > 1000
)

# Broadcast small dimension table
result = df1.join(
    broadcast(df2),
    "customer_id",
    "left"
)
```

Check the execution plan:

DATA ENGINEER

LinkedIn@AshishZope

```
result.explain(True)
```

Interview Point: A good answer should start with **diagnosis using Spark UI + execution plan**, followed by the appropriate optimization—not blindly changing cluster resources.



Most Important Topics to Prepare

For a **Senior Data Engineer / PySpark interview**, I would prioritize these scenarios:

Priority	Topic
★★★★★	Data Skew
★★★★★	Broadcast Join
★★★★★	Window Functions
★★★★★	Incremental Load
★★★★★	Partitioning
★★★★★	Shuffle
★★★★★	Performance Optimization
★★★★	Duplicate Handling
★★★★	Joins
★★★★	Nested JSON / Explode
★★★★	Caching & Persistence
★★★★	Repartition vs Coalesce
★★★★	Spark Architecture
★★★	UDF
★★★	RDD vs DataFrame

These 10 are particularly good for **scenario-based interviews** because the interviewer can keep asking “Why?”, “What happens internally?”, “How would you optimize it?”, and “What if the data is huge?” rather than just testing syntax.